ON THIS PAGE

What you are building
How a task runs
From idea to payout
Bundle layout
task.toml
How tasks are filed
Rules and limits
The difficulty bar
Writing the verifier
Authorship & originality
Validate locally
After you submit
Review & payment
Common mistakes
Glossary

Authoring tasks

This project collects expert-written tasks for benchmarks of hard, real computer work that today's best AI agents cannot do yet. Each task is a self-contained bundle in the harbor task format: an instruction, a containerized environment, and a sealed verifier that grades the result; some datasets also take a reference solution. The bar is high and the pipeline is strict. Read this page fully before your first submission. If harbor and benchmark tasks are new to you, start with the first two sections: they explain what you are making and how it runs. Every section is written for the dataset selected below.

PAYOUT PER APPROVED TASK

$300 every domain

Flat rate per approved task (USD). Human review happens after the automated pipeline gates pass. See Review & payment.

What you are building

A task is a job you would hand a strong colleague: a Linux machine set up with everything needed, a written brief, and a definite way to tell whether the job was done. An AI agent gets the same machine and the same brief, works on its own for a fixed time budget, and then a program you wrote checks the result. Benchmarks made of these tasks are how labs measure what agents can and cannot do, so a task is only useful when it is:

- Real. Work someone would be paid to do on a computer: science, software, ML, business operations, security, hardware, media. Lean into your strongest domain. A couple of hard tasks from deep expertise beat a pile of shallow ones.
- Beyond the frontier. An expert takes hours to days. Frontier agents fail for real reasons, not trivia, missing context, or artificial handicaps.
- Machine-checkable. A verifier runs after the agent finishes and scores exactly 0 or 1. A solution passes if and only if it completes the instruction.
- Fair with the internet open. Agents keep their full capabilities, network included. Blocking access is not a source of difficulty, and the solution to your task must not be findable online.

How a task runs

Every attempt at your task, on your laptop or in our pipeline, follows the same steps. Knowing them tells you which file does what and who can see it.

1. Harbor builds `environment/Dockerfile` into an image and starts a container from it. This is the agent's whole world: the operating system, the tools, and any data you COPY in.
2. The agent reads `instruction.md` and gets a shell in that container. It can run anything, install anything, and use the network. It works until it decides it is done or `[agent].timeout_sec` runs out.
3. The container stops. Harbor copies out the files listed in `artifacts` at the top of `task.toml`. Nothing else the agent wrote survives this step.
4. A second container is built from `tests/Dockerfile` and `tests/test.sh` runs inside it with the artifacts mounted. The agent never sees this container. The script writes `/logs/verifier/reward.txt` containing 0 or 1, plus a test report.

Two other runs use the same machinery. The oracle run replaces the agent with your `solution/solve.sh`, executed in the environment container, and must score 1. The nop run does

nothing at all and must score 0. Together they prove the task is solvable as written and that the verifier does not pass an empty attempt.

From idea to payout

1. Check the idea first (recommended). Paste it into the idea check on the Submit tab. It scores the idea against the acceptance rubric (difficulty, verifiability, specificity) and flags dead ends before you spend hours on a bundle. It is advisory and does not affect your submission.
2. Build the bundle in the layout below: instruction, environment, reference solution, sealed verifier, metadata.
3. Validate locally. The oracle must score 1, the nop must score 0, and the quality check should pass. The commands are under Validate locally.
4. Submit the zip here with the task's domain and labels. Structural checks run instantly.
5. The automated pipeline runs: AI check, similarity, reference verification on the real harness, an agentic quality review, an anti-cheat probe, a multi-attempt difficulty probe, and a run audit. You watch every stage live on your task page.
6. A reviewer makes the final call. Rejections include a written reason. Approved tasks are paid.

SCIENTIFIC COMPUTING

Pick Scientific computing on the submit form for tasks drawn from real research workflows in the natural, mathematical and engineering sciences: something a working scientist would be paid to do with a computer, not a course exercise or generic coding dressed in scientific vocabulary. The pipeline is the same. The structure check holds the bundle to a stricter, science-specific contract.

What reviewers look for

Home
Chat 1
Apply
Training
Payments
Surveys
Referrals
Documents

- Real data, or a disclosed, seeded generator whose ground truth comes from an independent source, never from the reference solution.
- Graded values that beat trivial baselines.
- Content that belongs to the declared field, from an author who practises it.
- A clear statement of whether the data is real or synthetic in `difficulty_explanation`, and of where the ground truth comes from in `verification_explanation`.

Bundle layout

Submit one zip of the task directory contents. The structure check requires `instruction.md`, `task.toml`, `environment/Dockerfile`, `solution/solve.sh`, `tests/test.sh` and `tests/Dockerfile`, allows only the entries shown at the bundle root, and expects a `README.md` with Difficulty, Reference solution and Verification sections.

```
your-task/
├── instruction.md           # the complete task instruction, written by you
├── task.toml                # template below
├── README.md                # ## Difficulty · ## Reference solution · ## Verificatio
├── environment/
│   ├── Dockerfile           # the agent's container
│   └── data/  ...           # optional, COPY'd into the image
├── solution/
│   └── solve.sh             # reference solution; must score 1 (required)
├── tests/
│   ├── test.sh              # writes /logs/verifier/reward.txt (exactly 0 or 1)
│   ├── Dockerfile           # bakes /tests, pins pytest==8.4.1 + pytest-json-ctrf==0
│   └── ...                  # ground truth, sealed from the agent
└── authoring/               # optional, never mounted into a container
    ├── provenance/  ...     # generators, seeds, source manifests: where the data ca
    └── evidence/  ...       # cheat attempts, notes, validation records
```

Home
Chat 1
Apply
Training
Payments
Surveys
Referrals
Documents

The environment folder becomes the agent's container. `solution/` is mounted only for the oracle run. `tests/` is baked into a separate verifier image the agent never sees. Nothing in `environment/` may reference solution or test files; that leaks answers and fails validation.

`authoring/` is never mounted into a container. Keep generators, seeds, source manifests and your own cheat attempts there.

task.toml

Copy this template and fill in every field. Every `[metadata]` key shown is required except `research_advisor` and `referred_by`. Keep `category` at Science and set `subcategory` to the field's display name.

```
schema_version = "1.4"

artifacts = [] # paths the verifier reads from the agent; must be above the first [sec

[task]
name = "afterquery/<your-task-slug>" # must equal the task name you submit
description = "" # one sentence: what the task asks for
authors = [{ name = "", email = "" }] # mirrors author_name / author_email
keywords = [] # mirrors tags

[metadata]
author_name = ""
author_email = ""
author_organization = "" # your university, institute, lab or employer, or exactly "In
author_profile = "" # Google Scholar link (LinkedIn or a personal site also works)
research_advisor = "" # optional: your advisor/supervisor, if a PhD student or postdoc
referred_by = "" # optional: who referred you, if anyone
conflicts_of_interest = "None" # disclose any (employer, vendor engagement, competing
domain = "" # life-sciences | physical-sciences | earth-sciences | mathematical-scienc
field = "" # one of the four fields in the domain (table below), e.g. "astronomy"
```

Home
Chat 1
Apply
Training
Payments
Surveys
Referrals
Documents

```
subfield = "" # free text: your specialization within the field, e.g. "radio interferc
category = "Science" # keep exactly this
subcategory = "" # the field's display name, e.g. "Astronomy"
tags = [] # specific, relevant keywords; at least one
expert_time_estimate_hours = 0 # best-case hours for a focused domain expert (must be
relevant_experience = "" # your professional background for THIS task; generic stateme
difficulty_explanation = "" # why this task is hard for agents and humans; say whether
solution_explanation = "" # high-level approach and key insights
verification_explanation = "" # how the tests verify correctness, and where the ground

[verifier]
timeout_sec = 1800.0 # increase based on test suite runtime (max 13500)
environment_mode = "separate" # verifier runs in its own container (required)

[verifier.environment]
network_mode = "no-network" # required; the verifier runs offline

[agent]
timeout_sec = 9000.0 # min 9000 (2.5h), max 28800; research workflows routinely need h
# user = "agent" # optional: run the agent as this non-root user (keep environment/Doc

[environment]
build_timeout_sec = 600.0
cpus = 1 # standard sizes only: 1 / 2 / 4 / 8 / 16
memory_mb = 2048 # standard sizes only: 1024 / 2048 / 4096 / 8192 / 16384
storage_mb = 10240 # max 40960
gpus = 0 # 0, or exactly 1 with the GPU option; any field may ask for one
# gpu_types = ["H100"] # required, exactly one class, when gpus = 1
network_mode = "public" # required; agents keep the open internet
# never set allow_internet; the deprecated key fails validation
```

`[task].name` must be `afterquery/<your-task-slug>`, where the slug equals the task name you submit: lowercase kebab-case, at most 3 hyphen-separated words.

Home
Chat 1
Apply
Training
Payments
Surveys
Referrals
Documents

How tasks are filed

Pick the domain, then the field the task's core computational work belongs to. Your task.toml must carry the same `domain` and `field` slugs plus a free-text `subfield`. Fields are broad umbrellas: the slug reads narrower than what it covers.

Cross-cutting work files by its dominant content. Nuclear neutronics is physics, thermal-hydraulics is mechanical engineering, fuel cycle is chemical engineering. `chemical-engineering` is process, reactor and transport at scale; `chemistry` is molecular science.

DOMAIN	FIELDS, BY THE TASK'S CORE COMPUTATIONAL WORK
Life Sciences `life-sciences` Biology, ecology, medicine, neuroscience — genomics and sequence analysis, population modeling, clinical data, neural data pipelines.	**Biology & Biotechnology** `biology`: bioimaging, genomics, synthetic biology **Ecology & Evolutionary Biology** `ecology`: phylogenetics, population genetics **Medicine & Health Sciences** `medicine`: clinical, epidemiology, public health **Neuroscience & Cognitive Science** `neuroscience`: neuroimaging, psychophysics, behavior
Physical Sciences `physical-sciences` Astronomy, chemistry, materials science, physics — survey photometry and spectra, molecular modeling, crystal-structure workflows, physical simulation.	**Astronomy & Astrophysics** `astronomy`: cosmology, instrumentation, time-domain **Chemistry** `chemistry`: molecular science **Materials Science & Engineering** `materials-science`: characterization, computational materials **Physics** `physics`: simulation, numerical modeling, inverse design

DOMAIN	FIELDS, BY THE TASK'S CORE COMPUTATIONAL WORK
Earth Sciences `earth-sciences` Atmospheric, environmental, geo- and ocean sciences — climate and weather model output, environmental monitoring, seismic and geospatial analysis, ocean circulation.	**Atmospheric & Climate Sciences** `atmospheric-sciences`: climate modeling, meteorology **Environmental & Sustainability Sciences** `environmental-sciences`: energy systems, pollution, land use **Geosciences & Planetary Science** `geosciences`: seismology, hydrology, planetary **Ocean & Marine Sciences** `ocean-sciences`: physical & biological oceanography
Mathematical Sciences `mathematical-sciences` Applied and formal mathematics, operations research, statistics — numerical methods, proof assistants, optimization and scheduling, statistical inference.	**Applied Mathematics & Scientific Computing** `applied-mathematics`: numerics, PDEs, HPC **Formal Mathematics & Theorem Proving** `formal-mathematics`: Lean, Coq, autoformalization **Operations Research & Optimization** `operations-research`: convex & combinatorial optimization, scheduling **Statistics & Machine Learning** `statistics`: inference, ML methods, probabilistic modeling
Engineering Sciences `engineering-sciences` Chemical, civil, electrical, mechanical engineering — process simulation, structural analysis, circuits and signals, CAD/CAE and control.	**Chemical & Biomolecular Engineering** `chemical-engineering`: biomolecular, process, energy **Civil & Structural Engineering** `civil-engineering`: structural, geotechnical, transportation **Electrical & Computer Engineering** `electrical-engineering`: computer hardware, signals, control, power **Mechanical & Aerospace Engineering** `mechanical-engineering`: aerospace, robotics, thermofluids

Rules and limits

The structure check runs the moment you submit and rejects the bundle on any of these. The pipeline never starts on a bundle that fails one.

Home
Chat 1
Apply
Training
Payments
Surveys
Referrals
Documents

- Pin every pip and uv install in every manifest with `==`. The verifier pins are `pytest==8.4.1` and `pytest-json-ctrf==0.3.5`.
- Never version-pin apt packages; they go stale. Run `apt-get update` before installing and finish with `rm -rf /var/lib/apt/lists/*`.
- Set `[environment] network_mode = "public"` and `[verifier.environment] network_mode = "no-network"`. Never set `allow_internet`; the deprecated key fails validation.
- Give the agent at least 9000s (2.5h) and at most 28800s; the verifier timeout caps at 13500s and has no floor. cpus must be 1, 2, 4, 8 or 16. memory_mb must be 1024, 2048, 4096, 8192 or 16384. storage_mb is at most 40960. No `FROM --platform=` pins.
- `gpus` is 0, or exactly 1 when you pick the GPU option on the submit form; any field may ask for one. With a GPU, pin exactly one class with `gpu_types = ["H100"]`.
- Fill in your affiliation, profile link and conflict-of-interest disclosure. Placeholder values fail the author-facts check.
- Do not include eval-set canary markers (the `harbor-canary` GUID lines some public tasks carry). Tasks submitted here are not part of an existing public eval set, and the structure check rejects a bundle that contains one.
- Everything runs in one container: no `docker-compose` file. Install any service the task needs in `environment/Dockerfile` and ship a script that starts it; the instruction tells the agent to run it. If grading needs the service, `tests/test.sh` starts its own copy in the verifier container.
- File permissions from the upload are not kept. If `tests/test.sh` or `solve.sh` runs a compiled program from `tests/` or `solution/`, `chmod +x` it in that script first.
- End `environment/Dockerfile` as root. To run the agent as a non-root user, set `user = "<name>"` under `[agent]` in task.toml instead of a final `USER` line.
- Write `instruction.md` as plain prose: no Markdown headings.
- Keep `OMP_NUM_THREADS` and `MKL_NUM_THREADS` at or below the cpus declared for that image.
- Numeric checks reject NaN and infinity before comparing: `abs(x - truth) > tol` is never true for NaN, so check `math.isfinite(x)` first.

- Set each tolerance from the spread of correct methods, not the reference's own margin, and make every graded case and stated rule count: a constant, an average, or a solver that skips a stated rule must fail.
- Do not grade on kernel accounting (`VmHWM` , `smaps_rollup` or `NoNewPrivs` from `/proc` , or `st_blocks`), and do not leave files the tests or the solution read in `/tmp` at build time.

The difficulty probe runs 8 independent attempts. The task must be solved at least 0 and at most 7 times.

A verifier image that satisfies the rules above:

```
FROM python:3.13-slim-bookworm
RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/l
RUN pip install pytest==8.4.1 pytest-json-ctrf==0.3.5
COPY . /tests/
RUN mkdir -p /app
```

The difficulty bar

We run independent frontier-agent attempts at your task's full time budget. The task must be solved at least once and at most a set number of times; the exact band is under Rules and limits. Solved more often, it is below the bar. Solved never, it is treated as unsolvable as specified. Both fail.

- Aim beyond the frontier, honestly. The task should be hard because the underlying work is hard: the kind an expert spends hours to days on across several sessions. Difficulty must never be arbitrary. No unnecessary complexity, no trivia or memorization tests, no tokenizer or obfuscation tricks, no withholding information an expert would obviously have.
- Do not task-hunt against current models. Filtering piles of candidate ideas for whatever today's models happen to miss finds shallow gaps that vanish with the next release. Build tasks whose difficulty you can explain.

- Proven sources of difficulty: long-horizon work with many dependent steps; richer environments (microservices, filesystems, databases) the agent must explore; dynamic environments such as live traffic in a multi-container task; cross-domain tasks that need expert knowledge in several areas; iterative problems solved by trial and error rather than read-and-solve.
- The internet is open. Obscurity is not difficulty. If the solution can be found online, agents will find it.

Writing the verifier

The verifier is the half of the task most first-time authors get wrong. It decides whether an attempt counts, so it has to be strict, deterministic, and impossible to satisfy without doing the work. The pipeline enforces static checks at submit and a 35-criterion agentic rubric review afterwards. The rules that catch most bundles:

- Binary reward. `tests/test.sh` writes `/logs/verifier/reward.txt` containing exactly `0` or `1`, plus a CTRF test report (run pytest with the `pytest-json-ctrf` plugin). Every code path, including crashes, must produce a reward. Partial credit does not exist.
- Separate verifier. `[verifier] environment_mode = "separate"` is required. The verifier runs in its own container, built from `tests/Dockerfile`, after the agent's container is torn down. It can read only the files declared in the top-level `artifacts = [ ... ]`, and whatever `tests/Dockerfile` bakes in. Bake tests with `COPY . /tests/`, pre-install all verifier tooling in the image (never install inside test.sh), and `RUN mkdir -p` the parent directory of every declared artifact. The verifier image template is under Rules and limits.
- Demand evidence of real work. Ground truth and grading logic live only in `tests/`, never in the environment image. Check outputs, not exit codes or agent-writable state, and compare against values the agent could not have guessed.
- Use absolute paths in the instruction (for example `/app/output.json`) and name every expected output file explicitly. If the verifier reads it, the instruction must say so.

- Instruction suffix. `instruction.md` must end with a blank line, then exactly this sentence (N is the integer value of `[agent].timeout_sec`), then at most one trailing newline:

```
You have N seconds to complete this task. Do not cheat by using online solutions or hi
```

- One container. A task has no compose file and no sidecars: a service the agent needs (a database, a queue) is installed in the environment image and started by a script the instruction points to, and a service the verifier needs is started by `tests/test.sh` inside the verifier container.
- Try to cheat your own task. Ask what the laziest attempt that passes would be, then make the tests reject it. Do not ship the attempt: a bundle with a `cheat/` directory fails the structure check. The pipeline runs a dedicated adversarial probe that tries the same thing.

Authorship & originality

- Write the instruction yourself, as a domain expert, in your own words. We run an AI check on every instruction file, and flagged submissions are rejected. AI assistance elsewhere (environment scaffolding, test data generation) is fine. The instruction is the part that must be yours.
- Submit original work. Every submission is compared against the benchmark's public task set and all other submissions. A task that duplicates an existing one is rejected, including a rebuild of your own earlier task under a new domain. Resubmitting a byte-identical bundle is blocked outright.
- Each task must be a new problem against your own back catalogue, not only new subject matter. Swapping the domain nouns, renaming the files, and rewording the brief around the same scaffold, the same reference solution and the same verifier shape is a rebuild, and it is rejected however thoroughly the surface was changed. The more tasks you have built on one pattern, the further the next one has to be from them.

- Target a different failure mode each time. A task earns its place by breaking frontier models in a way the rest of the set does not. Before you build, ask what specifically the agent will get wrong: a race it cannot see, an invariant it will not preserve, a rule it has to infer rather than read. Check that it is not the same thing your last few tasks tested. Ten tasks that all reduce to reconstructing the exact numbers a missing tool would have produced are worth about as much as one, whether they are set in DNS caching, freight tariffs or gene networks.

Validate locally

Install harbor (`uv tool install harbor`) and Docker, then run all three checks from your task directory before submitting. The pipeline runs the same gates on the real harness, so a local failure is a guaranteed pipeline failure.

```
# from the task directory, with docker running:
harbor run -p . -a oracle -e docker    # reference solution must score exactly 1
harbor run -p . -a nop -e docker       # doing nothing must score exactly 0
harbor check . -m anthropic/claude-opus-4-8   # LLM review against the implementation
```

The first two commands print a reward at the end of the run. If the oracle scores 0, read the verifier log before touching the solution: most first failures are an artifact path the verifier cannot find, or a tool that was never installed in the verifier image. If the nop scores 1, the verifier is passing on nothing and has to change.

After you submit

Every submission runs the full pipeline before human review, and you watch each stage live on your task page, including agent trajectories as they stream in.

1. Structural checks: required files, task.toml policy, verifier isolation, blocked terms. Instant feedback at submit.

Home
Chat 1
Apply
Training
Payments
Surveys
Referrals
Documents

2. AI check: the instruction file is screened for AI-generated text.
3. Similarity: your task must not duplicate anything in the public task set or in what has already been submitted, and must be a new problem relative to your own earlier tasks rather than one of them re-skinned.
4. Reference verification: your reference solution and the untouched environment run repeatedly on the real harness. The oracle must score 1 every time and the nop 0 every time.
5. Quality review: an agentic reviewer reads your entire bundle against the implementation rubric: instruction completeness, verifier rigor, determinism, environment hygiene, metadata quality. Blocking criteria reject the task; advisory ones are surfaced to the reviewer.
6. Anti-cheat probe: an adversarial agent is explicitly invited to cheat your verifier. If it passes verification without doing the real work, the task fails.
7. Difficulty probe: independent frontier-agent attempts at the full time budget. The task must land in the solve band. Per-trial runtime, tokens, and cost are visible live, and you can cancel a run from the task page.
8. Run audit: a judge model reads the probe trajectories and checks for reward hacking, specification problems, and refusals. Legitimate depth passes; unfair specs and grader exploits fail.
9. Human review: a reviewer makes the final call. Rejections include a written reason.

Failures are classified. Verdict failures are about the task. Infra failures are platform flakes, never count against you, and get re-run.

Review & payment

Human review happens after the automated pipeline gates, not instead of them. Every submission runs the full pipeline first. Only once those gates pass does a reviewer read the task and make the final call. Rejections include a written reason.

Home
Chat 1
Apply
Training
Payments
Surveys
Referrals
Documents

Approved tasks on this dataset pay a flat **$300** per approved task, whatever the domain (also shown in the banner above).

Payment is recorded at approval and paid on the normal cycle. Resubmitting an unchanged bundle is blocked by content hash, and near-duplicates of anyone's task, including your own live ones, are rejected by the similarity gate.

Volume is not the goal. A batch of variations on one pattern will not clear it, and the further into a pattern you go the less likely the next one is to land. Five tasks that break models five different ways are worth far more to us than fifty that break them the same way.

Submit a task

Common mistakes

The failures we see most from first bundles, in rough order of frequency:

- A verifier that checks an exit code, a file's existence, or a string the agent can print, instead of the content of the result.
- Ground truth copied into `environment/`, or a test data file the agent can read. Anything in the environment image is visible to the agent.
- An instruction that never names the output path, or names a relative one. The verifier and the instruction must agree on absolute paths.
- A verifier that installs packages at run time. Bake every tool into `tests/Dockerfile`; the verifier must work with nothing but the image and the artifacts.
- Timeouts copied from the template. Time the oracle run and give the agent several times that. Hard tasks routinely need hours.
- An environment that depends on a live web resource that can change or disappear. Vendor the data into the image, or pin an immutable source.

- Nondeterministic grading: random seeds, wall-clock dependence, floating-point comparisons without a tolerance. Re-run the verifier on the oracle output several times. It must pass every time.
- An instruction that reads like a checklist of steps. State the goal and the deliverable and let the agent choose the route.
- A task whose difficulty is volume. Long is fine when each step needs judgement. Long because there are 400 files to touch is not.
- Metadata written in a hurry. Reviewers read the three explanations and `relevant_experience` first, and generic text there fails review.

Glossary

harbor	The open-source harness that builds and runs tasks. It defines the task format, runs agents against tasks, and produces the reward.
bundle	The zip of your task directory that you submit here.
environment	The Docker image the agent works in, built from environment/Dockerfile.
instruction	instruction.md, the brief the agent reads. It is the only thing the agent is told.
artifact	A file path listed in task.toml that harbor copies out of the agent's container for the verifier.
verifier	The tests/ container and script that grade an attempt and write the reward.
reward	The single 0 or 1 in /logs/verifier/reward.txt.
oracle	A run where your solution/solve.sh stands in for the agent. It must score 1.
nop	A run where nothing is done. It must score 0.



Home

1
Chat

Apply

Training

Payments

Surveys

Referrals

Documents



attempt, trial	One agent run at your task. The difficulty probe runs several.
trajectory	The full record of what an agent did during an attempt: commands, outputs, reasoning. You can read them on your task page.
rubric	The written criteria the agentic quality review and the human reviewers grade against.
verdict vs infra	A verdict failure is about your task. An infra failure is a platform problem; it is re-run and never counts against you.